ОФИЦИАЛЬНАЯ ИНЖЕНЕРНАЯ СПЕЦИФИКАЦИЯ & АРХИТЕКТУРНОЕ РУКОВОДСТВО

Solace Engine: Руководство по Архитектуре и Техническая Спецификация Разработки AA 3D Action-RPG

Полное техническое руководство по ядру движка C++20, графическому конвейеру C+, командному ИИ отряда, физическому контуру и производственному AA-пайплайну

ГЛАВА 1. ОБЩЕЕ ВИДЕНИЕ И АРХИТЕКТУРНЫЙ МАНИФЕСТ

Solace Engine — это специализированный графический и игровой движок, разрабатываемый на языке C++20 с использованием графического API OpenGL (Core Profile 3.3+/4.x) и мультимедийной библиотеки SDL3. Проект спроектирован для поддержки хардкорной 3D Action-RPG в жанре темного фэнтези с высокой степенью выживания, глубоким билдостроением и масштабными битвами уровня «Войны одного человека» (Power Fantasy).

Фундаментальные Столпы Проекта

- **Отсутствие кнопки-спама (No Button Mashing):** Игровой процесс строится вокруг таймингов, оценки ордера противников, управления выносливостью и пространственных физических микро-решений.
- **Физика векторных импульсов вместо i-frames:** Рывки и уклонения (дэши) представляют собой физические импульсы движения с инерцией массы. Кадры абсолютной неуязвимости (i-frames) полностью отсутствуют.
- **Адаптивная роевая сеть ИИ (Squad Director):** Противники функционируют как единый слаженный отряд под управлением глобального оркестратора, отслеживающего стиль игрока и распределяющего слоты атак через `Token Bucket System`.
- **Гибридная Графическая Архитектура C+:** Сочетание гибкой ECS-парадигмы с оптимизированным рендерингом (двухпроходная очередь `RenderQueue`, централизованный пул костей `BoneBufferPool` и Frustum Culling).

Структура Технического Руководства

Настоящий документ содержит исчерпывающее руководство для разработчиков и включает 5 ключевых разделов:

1. **Главы 1–3:** Концептуальное видение, 7-этапная модель прогрессии и физический боевой контур.
2. **Главы 4–6:** Командный ИИ, система восприятия, Squad Director и профилирование игрока.
3. **Главы 7–9:** Архитектура рендеринга C+, инкапсуляция OpenGL, Material batching и пул костей.
4. **Главы 10–12:** Детализированный фазовый план миграции, структура файлов C++20 и реактивный аудио-движок.
5. **Главы 13–15:** Производственный AA-пайплайн, дерево решений, матрица рисков и регламент верификации.

ГЛАВА 2. 7-СТУПЕНЧАТАЯ МОДЕЛЬ ПРОГРЕССИИ (ПУТЬ ОТ НОВИЧКА К МАСТЕРУ)

Динамика игрового процесса Solace Engine рассчитывается на постепенное изменение масштаба столкновений, поведения ИИ и доступного арсенала игрока. Ниже приведена полная официальная спецификация всех 7 этапов прогрессии.

ЭТАП	ПРОФИЛЬ ВРАГОВ	ДИНАМИКА И КЛЮЧЕВЫЕ МЕХАНИКИ	СОСТОЯНИЕ ИГРОКА
1. Шоковая Адаптация	Дикие звери, примитивные существа, базовые слизи.	Высочайшая летальность. Медленные анимации, жесткий дефицит сил. Выманивание целей по одной, использование ловушек и рельефа.	Уязвимый выживший
2. Порог Цивилизации	Мелкие мародеры, бандитские группы.	Появление примитивного оружия. Враги зажимают с двух сторон (щит с фронта, выпад с фланга). Освоение узких проходов и арок.	Начинающий боец
3. Тактическое Пробуждение	Организованные наемники, мечники со щитами.	Появление векторных дэшей. Чтение микро-телеграфов позы врага вместо UI-подсказок. Наказание противника в фазах recovery.	Закаленный тактик
4. Адаптивный Перелом	Смешанные отряды (лучники, маги, щитоносцы).	ИИ активирует динамическую память (`Squad Blackboard`) и систему Player Profiling. Начало осознанного билдостроения.	Опытный командир
5. Профессиональный Строй	Армейские когорты, фаланги с башенными щитами.	Высокие скорости боя. Использование комбо-цепочек, ломка вражеской стойки тяжелыми импактами, контроль средней дистанции.	Мастер клинка
6. Катаклизм	Легионы элиты (50+), массовые волны противников.	Полномасштабные столкновения. Режим Overclock (замедление восприятия), ультимативные способности, деформация окружения.	Живое бедствие
7. Сингулярность	Боссы-зеркала с симметричными навыками.	Финальное испытание. Высокоскоростной поединок с противниками, использующими расчетные дэши, микро-парирование и перехват.	Дуэль равных

Подробный Анализ Фаз Трансформации Геймплея

Ранняя Игра (Этапы 1–2): Игрок действует в условиях экстремального дефицита ресурсов. Попадание в окружение из двух мобов является смертельным сценарием. Основная тактика — стелс, привлечение внимания шумом и узкие проходы.

Средняя Игра (Этапы 3–5): Геймплей смещается в сторону тактического фехтования. Игрок осваивает векторные рывки и микро-парирование. Противники объединяются в отряды и начинают использовать фланговые обходы.

Поздняя Игра (Этапы 6–7): Переход в режим «Войны одного человека». Накопление Шкалы Потока позволяет рассекать десятки врагов за выпад, активировать Bullet-Time и ломать элементы архитектуры.

ГЛАВА 3. БОЕВОЙ КОНТУР И ФИЗИЧЕСКАЯ МОДЕЛЬ (`COMBAT_SYSTEM.CPP`)

В `Solace Engine` закладывается физически честный боевой контур, полностью исключающий условности аркадных файтингов.

3.1. Прецизионные Хитбоксы (Swept-Hitboxes)

Оружие не использует стандартные сферические или боксовые триггеры. Траектория лезвия непрерывно рассчитывается между кадрами t_{-1} и t_0 как заметаемый объем (Swept-Volume):

```
// Расчет векторов движения лезвия в combat_system.cpp Vec3 top_curr = sword.getTipPosition(); Vec3 top_prev = sword.getPreviousTipPosition(); Vec3 base_curr = sword.getBasePosition(); Vec3 base_prev = sword.getPreviousBasePosition(); // Формирование заметаемой плоскости и проверка пересечения с AABB/Obb врагов RayRayIntersectionResult result = TraceSweptSegment(top_prev, top_curr, base_prev, base_curr); if (result.hasHit) { ApplyImpactImpulse(result.entity, result.hitPoint, result.normal); }
```

> **Пересечение с геометрией окружения:** При контакте заметаемой плоскости атаки со стеной, колонной или башенным щитом врага клинок мгновенно останавливается. Запускается фаза отдачи (Bounce-Back) и персонаж входит в неперебиваемые фреймы восстановления (Recovery Frames).

3.2. Векторная Инерция Массы и Отказ от i-Frames

- **Импульс Рывка:** Рывок передает физический вектор скорости: $V_{new} = V_{current} + (F_{dash} / m_{armor}) \times \Delta t$
- **Отсутствие неуязвимости:** Во время исполнения рывка коллизия персонажа остается на 100% открытой для входящих swept-хитбоксов противников. Рывок служит исключительно для пространственного ухода с линии атаки.
- **Влияние экипировки:** Масса тяжелых доспехов увеличивает время разгона и длину тормозного пути, а также повышает расход выносливости.

3.3. Висцеральный Импакт (Hit-stop & Bone Squashing)

Hit-stop (Микрозаморозка): В момент контакта клинка с телом или броней обновляющие функции физики и анимации фиксируются на 2–8 фреймов для передачи плотности материала.

Bone Squashing (Процедурная деформация): Локальное сжатие костей скелета в точке попадания тупого оружия без сложной динамической резки меша.

ГЛАВА 4. СЕНСОРНЫЙ ДВИЖОК ИИ (PERCEPTION ENGINE)

Противники в Solace Engine не обладают скриптовым «всевидением» мирового состояния. Каждая единица ИИ наделена изолированными сенсорными модулями зрения и слуха.

4.1. Сенсор Зрения (Visual Sensor)

Обнаружение цели рассчитывается в реальном времени на основе трех параметров:

1. **Конус Видимости:** Проверка угла $\theta \leq 110^{\circ}$ между вектором взгляда юнита F и вектором на игрока D.
2. **Линия Взора (Raycasting):** Трассировка лучей от глаз ИИ к ключевым костям скелета игрока для детекции преград.
3. **Уровень Освещенности:** Нахождение игрока в густой тени сокращает дистанцию обнаружения с 20 метров до 4 метров.

4.2. Акустическая Разведка (Acoustic Sensor)

Любое активное физическое действие игрока генерирует в точке пространства событие акустического шума с заданным радиусом R:

ДЕЙСТВИЕ ИГРОКА	РАДИУС ШУМА (R)	РЕАКЦИЯ И ИЗМЕНЕНИЕ СОСТОЯНИЯ ИИ
Стелс-шаг по мягкому грунту	1.5 – 2.0 м	Игнорирование. ИИ остается в состоянии Patrol.
Обычный шаг / Бег по гравию	6.0 – 8.0 м	Поворот головы в точку шума. Переход в состояние Suspicious.
Физический рывок (Dash)	12.0 м	Мгновенный поворот корпуса, переход в режим Alert.
Удар оружием по стенке / щиту	25.0 м	Переход в состояние Combat Engaged. Сигнал тревоги отряду.
Разрушение деревянной конструкции	35.0 м	Глобальный призыв подкрепления ближайших патрулей.

4.3. Состояния Памяти Угрозы (Alert States)

ИИ проходит через 4 фазы реакции на раздражители: `Patrol` (Патрулирование) → `Suspicious` (Подозрение, проверка точки шума) → `Alert` (Поиск цели с обнаженным оружием) → `Combat Engaged` (Передача контекста в Squad Director).

ГЛАВА 5. ОРКЕСТРАТОР ОТРЯДА И СИСТЕМА ТОКЕНОВ (`SQUAD DIRECTOR`)

Враги в Solace Engine никогда не атакуют беспорядочно. Они объединены в общую сеть управления отрядом под контролем класса `SquadDirector`, синхронизирующего данные через общий контейнер состояния `SquadBlackboard`.

5.1. Динамические Роли в Отряде

- **Front Anchor (Заслон):** Удерживает прямой контакт с игроком, сковывает внимание и провоцирует на блоки.
- **Flanker / Assassin (Фланкер):** Мобильные юниты обходят по широкой дуге в слепые зоны камеры, выжидая фазу замаха игрока для удара в спину.
- **Suppressor (Подавитель):** Стрелки и маги перекрывают зоны маневра и отрезают пути отступления.
- **Disrupter (Перехватчик):** Юниты с башенными щитами, сбивающие тяжелые замахи игрока щитовым башем.
- **Commander (Тактик):** Отдает приказы перегруппировки и накладывает усиления.

5.2. Управление Агро через Token Bucket System

Чтобы избежать хаотичного наложения хитбоксов нескольких мобов и мгновенной нечестной смерти игрока, оркестратор выдает ограничение на число одновременно выполняемых атак:

```
// Концептуальный метод выдачи токенов в SquadDirector.cpp bool
SquadDirector::requestAttackToken(EntityID enemyID, AttackType type) { if (type ==
AttackType::Melee && m_availableMeleeTokens > 0) { m_availableMeleeTokens--;
m_activeTokens[enemyID] = type; return true; // Юнит получает право на удар } return false; // Юнит
обязан уйти во фланговое смещение }
```

> **Поведение юнитов без токенов:** Враги, не получившие токен атаки, не стоят на месте. Они постоянно сокращают и разрывают дистанцию, совершают ложные замахи (Feinting) и сжимают кольцо окружения, провоцируя игрока на ошибки.

5.3. Система Психологии и Слома Морали (Morale System)

Уничтожение командира отряда или мгновенный разгром передовой линии идеальным парированием вызывают у рядовых противников сбой координации, временный шок (Stun на 1.5–3 секунды) или паническое отступление.

ГЛАВА 6. ДИНАМИЧЕСКАЯ АДАПТАЦИЯ ИИ (PLAYER PROFILING)

Для исключения эффекта «нечестного чтения ввода» (input reading) ИИ использует систему накопительной статистической памяти, анализируя действия игрока за последние несколько столкновений.

6.1. Накопительные Метрики Сборки

Оркестратор `SquadBlackboard` отслеживает веса следующих действий игрока за окно времени $T = 30$ сек:

- W_{melee_spam} — Соотношение частых быстрых ударов ближнего боя.
- W_{ranged_magic} — Соотношение использования дистанционной магии и луков.
- W_{parry_heavy} — Успешность и частота использования микро-парирования.

6.2. Автоматические Контрмеры ИИ

ПРОФИЛЬ ИГРОКА	ПОВЕДЕНИЕ ИГРОКА	ТАКТИЧЕСКИЕ КОНТРМЕРЫ ИИ
Melee Heavy ($W_{melee} > 0.7$)	Частый спам быстрыми атаками в ближнем бою.	Передовая линия смыкается в глухой блок щитов. Юниты класса Disrupter активируют частые щитовые баши.
Ranged / Kiting ($W_{ranged} > 0.6$)	Кайтинг, отступление, дистанционная магия.	Фланкеры начинают агрессивный разгон по широкой дуге в слепые зоны. Стрелки перекрывают укрытия прижимным огнем.
Parry Master ($W_{parry} > 0.5$)	Частое использование микро-парирования в последний кадр.	Враги переходят на фейковые замахи (Feint Attacks), сбрасывая тайминг парирования и нанося удар в фазу recovery.

6.3. Задержка Перенастройки (Adaptation Window)

Перестройка строя ИИ занимает от **3 до 5 секунд** реального времени. Это дает игроку окно возможностей для своевременной смены стойки или использования ультимативных способностей до того, как ИИ выстроит контр-построение.

ГЛАВА 7. ГРАФИЧЕСКАЯ АРХИТЕКТУРА C+ И ИНКАПСУЛЯЦИЯ OPENGL

В рамках рефакторинга графической подсистемы происходит переход от архаичной модели прямых вызовов OpenGL к высокопроизводительному уровню **C+ Architecture**.

7.1. Ограничения Классического ECS (C) и Преимущества C+

Классический подход C (чистый ECS, где RenderSystem последовательно обходит сущности с RenderComponent) обеспечивал чистоту архитектуры, но приводил к критическим просадкам производительности:

- При 50 сущностях вызов 50 отдельных Draw Calls допустим.
- При 500 сущностях 500 вызовов Draw Calls (с дублированием переключений контекста GL, шейдеров и текстур) обрушивают бюджет кадра.

7.2. Фундаментальные Принципы Слоя C+

1. **Изоляция низкоуровневого API:** Вызовы OpenGL допускаются строго внутри `engine/graphics/renderer.cpp`. Греп-проверка `grep -rn "glBind|glDraw|glEnable" game/` должна возвращать пустой результат.
2. **Разделение принятия решений и рендеринга:** Класс `RenderQueue` только собирает и сортирует команды; `Renderer` — только выполняет вызовы API.
3. **Материал как сущность первого уровня:** Введение формализованного класса `Material` для группировки вызовов по состояниям конвейера.

> **Отказ от преждевременных абстракций:** Абстрактный интерфейс `IRenderer` не вводится до появления второго графического бэкенда (Vulkan или Direct3D). Работа ведется напрямую с монолитным классом `Renderer` для исключения накладных расходов виртуальных таблиц (vtable).

ГЛАВА 8. ДВУХПРОХОДНАЯ ОЧЕРЕДЬ `RENDERQUEUE` И МАТЕРИАЛЫ

Класс `RenderQueue` разделяет фазу сбора и сортировки геометрии от фазы непосредственного выполнения команд отрисовки.

8.1. Структура Вызова `DrawCall`

```
struct DrawCall { MeshResource* mesh; // Указатель на геометрию (VAO/VBO/EBO) Material* material;
// Указатель на материал (шейдер, текстуры, uniforms) Mat4 modelMatrix; // Матрица трансформации
объекта GLuint boneSSBO; // Идентификатор SSBO буфера костей (0 для статики) GLuint boneOffset; //
Смещение в пуле костей AABB bounds; // Ограничивающий параллелепипед для отсечения uint32_t
sortKeyShader; // Ключ сортировки шейдера uint32_t sortKeyMaterial; // Ключ сортировки материала };
```

8.2. Двухпроходная Сортировка Геометрии

Очередь разделяет все объекты на два вектора:

- **m_opaque (Непрозрачные объекты):** Сортируются *Front-to-Back* по глубине относительно камеры и группируются по материалам для минимизации переключений шейдеров и текстур.
- **m_transparent (Прозрачные объекты):** Сортируются *Back-to-Front* по удаленности от камеры для корректной альфа-сборки без артефактов глубины.

8.3. Жизненный Цикл Очереди на Кадре

Каждый кадр конвейер рендеринга проходит три обязательных этапа:

`RenderQueue::submit()` (Сбор DrawCalls от систем) → `RenderQueue::sort()` (Material Batching & Depth Sort) → `RenderQueue::execute()` (Передача данных в Renderer).

ГЛАВА 9. ЦЕНТРАЛИЗОВАННЫЙ ПУЛ КОСТЕЙ (`BONEBUFFERPOOL`) И INSTANCING

При рендеринге десятков скелетных персонажей создание и привязка отдельного SSBO буфера на каждый экземпляр создавала критический bottleneck. В архитектуре C+ введен класс **`BoneBufferPool`**.

9.1. Принцип Работы `BoneBufferPool`

Пул выделяет единый централизованный SSBO-буфер под суммарную емкость:

$$\text{Capacity} = \text{Capacity_Instances} \times \text{Bones_Per_Instance} \times \text{sizeof(Float4x4)}$$

```
// Привязка диапазона пула костей в Renderer.cpp void Renderer::bindBoneSlot(uint32_t slotIndex) { GLintptr offset = slotIndex * BONES_PER_INSTANCE * sizeof(Mat4); GLsizeiptr size = BONES_PER_INSTANCE * sizeof(Mat4); glBindBufferRange(GL_SHADER_STORAGE_BUFFER, BONE_BINDING_POINT, m_boneBufferPoolSSBO, offset, size); }
```

9.2. GPU Instancing Статического Окружения

Статические элементы декораций, деревья и пропсы с одинаковыми мешами объединяются в единый вызов **`glDrawElementsInstanced`**. Матрицы трансформаций загружаются в атрибуты экземпляров (Instance VBO).

9.3. Frustum Culling на Базе AABB

Раннее отсечение геометрии выполняет проверку пересечения ограничивающего параллелепипеда (AABB) объекта с 6 плоскостями конуса видимости камеры до формирования структуры **`DrawCall`**.

ГЛАВА 10. ДЕТАЛИЗИРОВАННЫЙ ПЛАН ПОЭТАПНОЙ МИГРАЦИИ

Миграция движка на целевую архитектуру C+ проходит через 5 атомарных фаз. Каждый этап завершается успешной сборкой, прохождением тестов и работающей игрой.

ФАЗА	СОДЕРЖАНИЕ И ЗАДАЧИ ЭТАПА	СРОКИ И ЗАТРАТЫ	УРОВЕНЬ РИСКА
Фаза 0	Удаление устаревшего кода (PlayerModel, Animator). Консолидация компонентов в **`player_components.hpp`**. Перенос **`camera.hpp`** в **`game/graphics/`**.	1–2 часа	Низкий
Фаза A	Вынос логики **`on_render`** из **`GameplayScene`** в **`RenderSystem`**. Введение структуры-бандла **`SceneRenderView`**.	2–3 часа	Низкий
Фаза B	Реализация классов **`Renderer`**, **`RenderQueue`** и структуры **`Material`**. Исключение метода **`draw()`** из класса Mesh.	1.5–2 дня (10–16 ч)	Средний
Фаза C	Полный отказ от ООП-класса Enemy. Переход на ECS entities. Реализация **`BoneBufferPool`** с привязкой **`glBindBufferRange`**.	4–6 вечеров	Высокий
Фаза C+	Frustum Culling по AABB, Material Batching в **`RenderQueue::sort()`**, Instancing статических пропсов.	3 вечера	Низкий–Средний

Технологии, Исключенные из Плана Миграции

- **Deferred Rendering (G-Buffer):** Стилизованный Toon/Rim шейдинг проекта эффективнее работает в рамках Forward-конвейера.
- **Compute Shaders для скининга:** При размере скелетов (21–34 кости) накладные расходы на Compute-шейдеры превышают выигрыш.
- **Переход на Vulkan:** Основной bottleneck находится на сторону CPU (вызовы отрисовки), что полностью решается архитектурой C+.

ГЛАВА 11. СТРУКТУРА ФАЙЛОВ И МОДУЛЕЙ ПРОЕКТА

Ниже приведена официальная полная древовидная структура исходного кода C++20 и ресурсов Solace Engine.

```
Solace/ ├── assets/ | ├── audio/ # 3D-звуки эффектов и мультитрековые стемы | ├── models/ # Стилизованные 3D-модели (.glb / .gltf) | └── shaders/ # PBR и Post-process shaders (GLSL) | ├── core/ # base_vs.glsl, base_fs.glsl, post_vs.glsl, post_fs.glsl | └── entities/ ├── engine/ # Статическая библиотека (SolaceEngine.lib) | ├── include/engine/ | | ├── ai/ # behavior_tree.hpp, enemy_modules.hpp, squad_director.hpp | | ├── audio/ # audio_system.hpp (4-channel Stems Engine) | | ├── core/ # engine.hpp, window.hpp, logger.hpp | | ├── graphics/ # renderer.hpp, render_queue.hpp, bone_buffer_pool.hpp, material.hpp | | ├── systems/ # combat_system.hpp, input_system.hpp, item_system.hpp | | └── utils/ # model_loader.hpp, math_utils.hpp | └── src/ # Реализация модулей движка └── game/ # Исполняемый файл (Solace.exe) ├── include/game/ | ├── entities/ # player_components.hpp, enemy_factory.hpp | └── world/ # world_gen.hpp, structures.hpp └── src/ # main.cpp, game.cpp
```

Разделение Сфер Ответственности Модулей

Модуль `engine/` содержит всю платформонезависимую графическую, физическую и ИИ логику и не имеет зависимостей от игрового кода. Модуль `game/` содержит фабрики сущностей, генерацию уровней и игровой цикл.

ГЛАВА 12. АВТОРСКИЙ РЕАКТИВНЫЙ АУДИО-ДВИЖОК (`AUDIO_SYSTEM.HPP`)

Звуковое сопровождение Solace Engine является интерактивным элементом геймдизайна, чутко реагирующим на смену боевых состояний и накопление Шкалы Потока.

12.1. 4-Канальная Мультитрековая Система (Stems Engine)

Музыкальное сопровождение каждой зоны состоит из 4 синхронных каналов (stems), микшируемых в реальном времени с привязкой к темпу (BPM Lock):

АУДИО-КАНАЛ (STEM)	МУЗЫКАЛЬНОЕ СОДЕРЖАНИЕ	УСЛОВИЯ АКТИВАЦИИ И МИКШИРОВАНИЯ
Layer 0: Ambient	Низкий перкуссионный гул, одиночные струнные.	Активен постоянно в фазах исследования и патрулирования.
Layer 1: Tension	Ритмичный басовый пульс, тревожная перкуссия.	Активируется при переходе ИИ отряда в состояние Alert / Suspicious.
Layer 2: Action	Тяжелые гитарные риффы, динамические барабаны.	Включается при наступлении активной фазы боя (Combat Engaged).
Layer 3: Overdrive	Симфонический хор, соло-гитара, тяжелый перегруз.	Активируется при достижении Flow Level 3 (Overclock Mode).

12.2. Пространственный 3D-Звук и Аудио-Телеграфы

Каждый опасный выпад ассасина или стрелка из слепой зоны камеры сопровождается пространственным аудио-символом (звон прямой пряжки, шелест тетивы). Это позволяет игроку совершать микро-парирование на слух.

ГЛАВА 13. ПАЙПЛАЙН РАЗРАБОТКИ АА 3D-ИГРЫ

Производственный пайплайн оптимизирован под реалии одиночной разработки (соло-разработчик) или микро-команды (2–3 человека). Главная задача — исключение производственного ада (Production Hell) за счет контроля объема контента.

Ключевые Принципы Производительности Пайплайна

1. **Стилизация вместо реализма:** Использование 3D Stylized PBR / Cel-Shading снижает требования к детализации сетки и текстур в 4–5 раз.
2. **Ретаргетинг Анимаций:** Применение единого стандартного Humanoid Rig (50–70 костей) для импорта и адаптации готовых ассет-паков.
3. **Модульный Геймдизайн:** Повторное использование систем ИИ (`SquadDirector`) и процедурная генерация тайлсетов комнат (`world_gen.cpp`).

Инструментарий и Технологический Строй

- **Движок:** Solace Engine (C++20, SDL3, Modern OpenGL).
- **3D-Моделирование и Риггинг:** Blender (экспорт в `.glb` / `.gltf`).
- **Текстурирование:** Substance 3D Painter / ArmorPaint (Стилизованные PBR-текстуры).
- **Аудио-Продакшн:** Reaper / FL Studio (Оркестровые библиотеки + гитарные процессоры).

ГЛАВА 14. МАТРИЦА УПРАВЛЕНИЯ РИСКАМИ (RISK MANAGEMENT)

Официальный регламент митигации технических и производственных рисков проекта Solace Engine.

ВИД РИСКА	ВЕРОЯТНОСТЬ	ВЛИЯНИЕ	СТРАТЕГИЯ МИТИГАЦИИ (РЕШЕНИЕ)
Scope Creep (Разрастание)	Высокая	Критическое	Жесткий отказ от некритичных механик. Разработка строго по 7-этапному плану.
Сложность Риггинга	Высокая	Высокое	Использование стандартизированного Humanoid Rig и ретаргетинга ассетов.
Просадка FPS (50+ мобов)	Средняя	Высокое	Внедрение LOD для ИИ и геометрии; ограничение физики токенами атак.
Недостаток 3D-контента	Средняя	Среднее	Акцент на стилизованный PBR, динамический туман и контрастный Color Grading.
Сложность OpenGL миграции	Низкая	Высокое	Атомарность фаз миграции (0–C+) с возможностью полного git revert коммита.

Регламент Git-Коммитов и Отката

Каждая из 5 фаз рефакторинга C+ выполняется в отдельной ветке. Коммит создается строго после успешной проходки всех чек-листов регрессии.

ГЛАВА 15. ЧЕК-ЛИСТ РЕГРЕССИИ И КРИТЕРИИ ПРИЕМКИ СРЕЗА

После выполнения любой фазы миграции или добавления подсистемы обязательна проверка по следующему регламенту.

Официальный Чек-Лист Проверки Регрессии

1. **Сборка Проекта:** Успешная компиляция через CMake в конфигурации Debug (`windows-vs-debug`) без ошибок и предупреждений.
2. **Проверка Базового Геймплея:** Перемещение (WASD), рывок, атака, преследование ИИ, изменение HP при уроне, переключение анимаций (Idle/Run).
3. **Проверка Горя перезагрузки (F5):** Корректность обновления параметров шейдера (`rimColor`) и разрешения FBO (`targetHeight`).
4. **Контроль Логов:** Полное отсутствие предупреждений *'Shader: uniform not found'* и ошибок графического API *'GL error'*.
5. **Создание Коммита:** Фиксация атомарного коммита в системе контроля версий Git.

> **Критерии Приемки Финального Вертикального Среза (Vertical Slice 3.0 Acceptance):**
> • Стабильные **60+ FPS** в разрешении 1080p/4K при одновременном рендеринге и обсчете 50 юнитов ИИ.
> • Корректная деформация геометрии при тяжелых ударах (подмена на осколки Debris).
> • Бесшовное микширование 4 музыкальных каналов (Layer 0 → Layer 3) при активации режима Bullet-Time.